

Unix File System Part II

Device Independent Start here 9/17/2025

UNIX is device independent.

- This means that we can use the same system calls to read from the keyboard as we do to read from a file, a pipe, a CD, magnetic tape (gulp! - at first I thought that if you find this device, a museum will want to contact you - **I was wrong - tape drives still have use**. There are tape cartridges that hold 15TB and robotic tape libraries of up to 278PB) They are low cost storage and energy efficient. They also last a long time. More than 20 years.
- This has some interesting implications. For example, many years ago I wrote an application to read data from CDs and tested it on a standard disk. No problem when I tried it on CD,
- You can also "trick" an application into writing to a file instead of to the screen. Why would we want to do this? We will do this .later

Device driver - A program that handles all the details of performing input and output to a specific device. Talk about writing a driver. A device driver must provide access to a device through the open, close, read, write, and ioctl calls.

Sync functions

As mentioned before, Linux/UNIX support caching. Writes to the disk may therefore be delayed. This can be even beyond the time in which the file is closed. There are a couple additional calls to force the data for a file to be written. It is even more of an issue for network drives. Some of the system calls are below:

fsync and fdatasync

```
#include <unistd.h>
```

```
int fsync(int fildes);
```

Flushes data associated with a specified file descriptor from disk cache and waits for this processing to complete. This can be very useful if you want to make sure that the data is safely on a disk or if you are dealing with a network drive for which you want others to be able to read the data. You can also open with the O_SYNC option. What is the difference?

```
int fdatasync(int fildes);
```

Same as fsync, but does not update the file attributes. E.g. modify time.

Both functions: return 0 if successful and -1 if fails.

sync and syncfs

```
void sync(void);
```

Tells the Kernel to queue all changed buffers for writing. Officially, it does not wait for all the writing to complete before returning. (Linux actually waits for the sync to finish - at least the last time I checked) **The system daemon, update, calls sync about every 30 seconds.** (Looking at the web, 30 seconds is consistently given as this time interval.) This tells us something about how long changed data can stay in cache.

```
int syncfs(int fd);
```

Tells the the Kernel to synchronize all modified data in the file system referenced by the open file descriptor fd.

This works even if multiple processes have the file open.

Note: there are more calls. (as always)

Additional File System Calls

The following are some additional useful UNIX system calls for working with files.

getcwd - get pathname of working directory

```
char *getcwd(char *buf, size_t size);
```

The `getcwd` function returns a pointer to the path-name of the working directory. The value of `size` must be at least one byte greater than the length of the pathname to be returned. **Older functions usually just returned a pointer to the data. Why the change?**

If `buf` is a NULL pointer, `getcwd` will obtain `size` bytes to hold the working directory using `malloc(3)`. In this case, the pointer returned by **getcwd** should be used eventually as the argument in a subsequent call to **free**. This is usually not a great approach.

chdir - set the working directory.

```
int chdir(const char *path);
```

This function sets the working directory. Can be useful to make sure that an application is running in the correct directory. Note: I won't talk about the return value if it is obvious. In this case -1 means error.

sysconf - get configurable system variables (POSIX)

Note: system parameters are referenced by a symbol. The symbol consists of upper case letters and underlines.

```
#include <unistd.h>
```

```
long sysconf( int name );
```

The `sysconf` function provides a method for the application to determine the current value of a configurable system limits or options.

The `name` argument represents the system variable to be queried. The following table lists a few of the system variables, which may be queried, and the corresponding value for the `name` argument. Note: there are a lot of these arguments. See the man page.

Variable	name	Value
ARG_MAX	_SC_ARG_MAX	Size of the argument list in bytes.
CHILD_MAX	_SC_CHILD_MAX	Maximum number of child processes
CLK_TCK are useful.	_SC_CLK_TCK	This is the number of clock ticks per second. Recall how clock ticks
OPEN_MAX	_SC_OPEN_MAX	Maximum number of open files.

Upon successful completion, the `sysconf` function returns the current variable value on the system.

If `name` is an invalid value, `sysconf` returns -1 and `errno` is set to indicate the reason. If the variable corresponding to `name` is not defined on the system, `sysconf` returns -1 without changing the value of `errno`.

SEE EXAMPLE ([sysconf.cpp](#)) The code was written using C style `printf` functions to remind you of these.

Note: `getconf CHILD_MAX` command can also be used to access these environmental variables.

Note: if you need to access data from directories, recall that there is a set of system calls that let you pull in information from directories. (We had talked about these earlier) [Why can't you do this with standard UNIX reads and writes?](#)

File representation

Two types of files. Ordinary files and special files. Data files and directories are ordinary files. Special files are represent character devices (like the keyboard) and ordinary file represent block devices such as disks, and FIFO pipes for inter-process communication.

Directories in Linux/UNIX are quite simple.

- They consist of a list of file names and inode numbers.
- The inode is a structure which contains a description of the file. It is maintained in an internal table that is also recorded on the disk.
- There is one inode for each file. What do you think would be in the inode? The inode is a structure that contains the following types of information: (It does contain more)
 - file size, location, device ID, owner, group
 - permissions, modification time (mention `touch` system call.), access time,
 - link count. Indicates the number of directory entries that point to this inode.
 - Location of the file: Linux has - 12 direct pointers, 1 single indirect, 1 double indirect, and 1 triple indirect. This tracks the location of data in the files. Will be discussed in class.
 - `ls -li` lists files with inode ID.

Question: If each block of disk storage is usually 4096 bytes and 4 byte disk pointers, how big can a file be? Note: I have seen block sizes as big as 64K Linux typically uses 4096. The pointers represent block offsets.

$49152 = 12 * 4096$ direct

$4,194,304 = 1024 * 4096$ (1024 pointers in a block) single indirect.

$4 \text{ gig} = 1024 * 1024 * 4096$ double indirect

$4 \text{ terabytes} = 1024 * 1024 * 1024 * 4096$ triple indirect.

Note: the block size in our Linux system is 4096. This is the default block size. Other UNIX systems have different block sizes. Usually bigger.

Note: the pointer don't have to be byte offsets. They could be block pointers. That would give us 16 terabytes.

See the link below for a more detailed description of inodes.

https://en.wikipedia.org/wiki/Inode_pointer_structure or <https://en.wikipedia.org/wiki/Inode>

Stat functions (We looked at this a little in an example program.)

stat, start, and fstat system calls. To get information about a file. For example the last time it was modified. For example, it is useful to know if a configuration file has been modified. **This can be very useful.** Definitely if you work for security. I will give an example of how this can be used in class.

```
#include <sys/types.h>
#include <sys/stat.h>

int stat( char *path, struct stat *buf )      Information for file with name path.

int lstat( char *path, struct stat *buf )     Information for symbolic link (later).

int fstat(int fd, struct stat *buf)          Information for open file with file descriptor is fd. Why use this one?
```

stat has the following format:

```
struct stat {
    dev_t    st_dev;      /* ID of device containing file */
    ino_t    st_ino;      /* Inode number */
    mode_t   st_mode;     /* File type and mode */
    nlink_t  st_nlink;    /* Number of hard links */
    uid_t    st_uid;     /* User ID of owner */
    gid_t    st_gid;     /* Group ID of owner */
    dev_t    st_rdev;     /* Device ID (if special file) */
    off_t    st_size;     /* Total size, in bytes */
    blksize_t st_blksize; /* Block size for filesystem I/O */
    blkcnt_t st_blocks;   /* Number of 512B blocks allocated */

    /* Since Linux 2.6, the kernel supports nanosecond
       precision for the following timestamp fields.
       For the details before Linux 2.6, see NOTES. */

    struct timespec st_atim; /* Time of last access */
    struct timespec st_mtim; /* Time of last modification */
    struct timespec st_ctim; /* Time of last status change */

    #define st_atime st_atim.tv_sec      /* Backward compatibility */
    #define st_mtime st_mtim.tv_sec
    #define st_ctime st_ctim.tv_sec
};
```

Note: timespec has at least the following members: tv_sec and tv_nsec which gives the time in seconds and nano seconds.

Note:

- see the utime function in the text to change the access and modify time. Also mention the touch command.
Note: there is no info on when the file was created.
- There is also a stat command that will display this data on the screen.

Lab2

Write a program that will accept up to 10 file names as command-line arguments. This program will display the names of any file that has been accessed or modified. You should test the files every 2 seconds. Note: if a file has been modified it has also been accessed, so there is no reason to report that a file has been accessed if it has been modified.

In class we will discuss how we can test this lab. (Note the touch command modifies the access and modify times. We will also talk about how to set up command line arguments.

Directories:

Contains file names and inode IDs. Directories do not contain the actual inode data. Where is it???? Why?

1. Much quicker to perform operations like moving files and changing names.

2. Directories are concise. Easier to fit in memory.
3. Allows one file to be referenced by in more than one place. In class I will give an example of why this is useful.

The **unlink** system call is used to remove an entry from a directory. **This is not the exactly same as deleting the file.** Why? -- The directory entry is different than the inode entry. The file will only be removed if there are no directories that contain it's inode. The format of the unlink follows:

int unlink(const char *path)

Description:

The unlink system call removes the entry for the file path from its directory. If this entry was the last link to the file, and no process has the file open, then all resources associated with the file are reclaimed. If, however, the file was open in any process, the actual resource reclamation is delayed until it is closed, even though the directory entry has disappeared. **This is a good way of supplying temporary files.** We will talk about this in class.

Return Values

Upon successful completion, a value of 0 is returned. Otherwise, a value of -1 is returned, and errno is set to indicate the error.

We will discuss in class using unlink to create temporary files.

Links

Links - a tool that basically allows a file to have more than one name. These names may be to allow the same file to be referenced directly from multiple directories. (They cannot be referenced across multiple file systems.) Implemented by having each file point to the same inode. ("ls -l" list links among other things). Note: since a parent directory is referenced by its children, it has multiple links.

ln command creates links. Form: ln original new. Note that there are options to this command. Do example using the following command:

```
echo "I am tired" > duck
```

```
ln duck dog
```

```
echo " and sleepy" > dog
```

You can view the inodes by typing **ls -li**. These are currently impressively large numbers. Note that the two files may share the same inode. For example, a parent and a child directory will both have an entry for the parent directory.

Note: there is also a "link" system call to do the same thing.

Note: there are also symbolic links. These are like aliases. The following creates a symbolic line between dog and duck: **ln -s duck dog**. You can use these for directories. Note what happens when you do a long listing. Note: there is a function to create links. We will show this with a directory listing.

Note: links are handy if you are creating libraries. We will talk about this in class. It can also create problems which we will also talk about. They are also great when you are generating data.

The fcntl (file control) and ioctl (io control) functions will be left until later. For example, you can use these calls to make a file descriptor non-blocking.